CHAPTER 3

Effective Web Services Design and Implementation

WEB services must interact with clients to receive their requests and return responses. In between the request and the response, a Web service applies appropriate business logic to process and fulfill a client's request. Designing an effective Web service starts with understanding the nature of the service to be provided—that is, how the service is going to interact with the client and how it is going to process and fulfill the client's request—coupled with how users of the service make their requests. This chapter examines Web services from the perspective of a service's interaction and processing functionality.

The chapter describes the key issues you must consider when designing a Web service, then shows how these considerations drive the design and implementation of a service's functionality. In particular, the chapter examines the interactions between a service and its clients and the business processing that the service performs. It illustrates these considerations by drawing examples using three typical Web service scenarios.

The chapter covers most of the decisions that must be made when designing and implementing a Web service, including identifying the different possibilities that give rise to different solutions. It describes how to receive requests, delegate requests to business logic, formulate responses, publicize a Web service, and handle document-based interactions.

Along the way, the chapter makes recommendations and offers some guidelines for designing a Web service. These recommendations, which appear in ital-

ics, include discussions of justifications and trade offs. They are illustrated with the example service scenarios.

3.1 Example Scenarios

Recall the Web services scenarios described in Chapter 1. Let's revisit these scenarios from the point of view of designing a Web service. Rather than discuss design issues abstractly, we'll use these typical scenarios to illustrate important design issues and to keep the discussion in proper perspective.

In this book, we focus on three types of Web services:

1. An informational Web service serving data that is more often read than updated—clients read the information much more than they might update it. A good example is a service that a client might use to obtain weather information for a given city.
2. A Web service that concurrently completes client requests while dealing with a high proportion of data that is updated frequently and hence requires heavy use of EIS or database transactions. An airline reservation system is a good example of this type of Web service. Many clients can simultaneously send details of desired airline reservations and the Web service concurrently handles and conducts these reservations.
3. A business process Web service whose processing of a client request includes starting a series of long-running business and workflow processes. A travel agency service is one such service. A service such as this receives the details of a travel plan request from a client, and then the service initiates a series of processes to reserve airlines, hotels, rental cars, activities, and so forth for the specified dates.

Discussions of Web service design issues in this chapter include references to these scenarios. However, the discussions use only appropriate characteristics of these scenarios as they pertain to a particular design issue, and they are not meant to represent a complete design of a scenario.

3.2 Structure of Web Services

In a Web service scenario, a client makes a request to a particular Web service, such as asking for the weather at a certain location, and the service, after processing the request, sends a response to the client to fulfill the request. When both the client and the Web service are implemented in a Java environment, the client makes the call to the service by invoking a Java method and receives as the response the result of the method invocation.

To give you the proper context for designing Web services, let's first take a high-level view at what happens beneath the hood in a typical Web services implementation. Figure 3.1 shows how a Java client communicates with a Java Web service interface on the J2EE 1.4 platform.

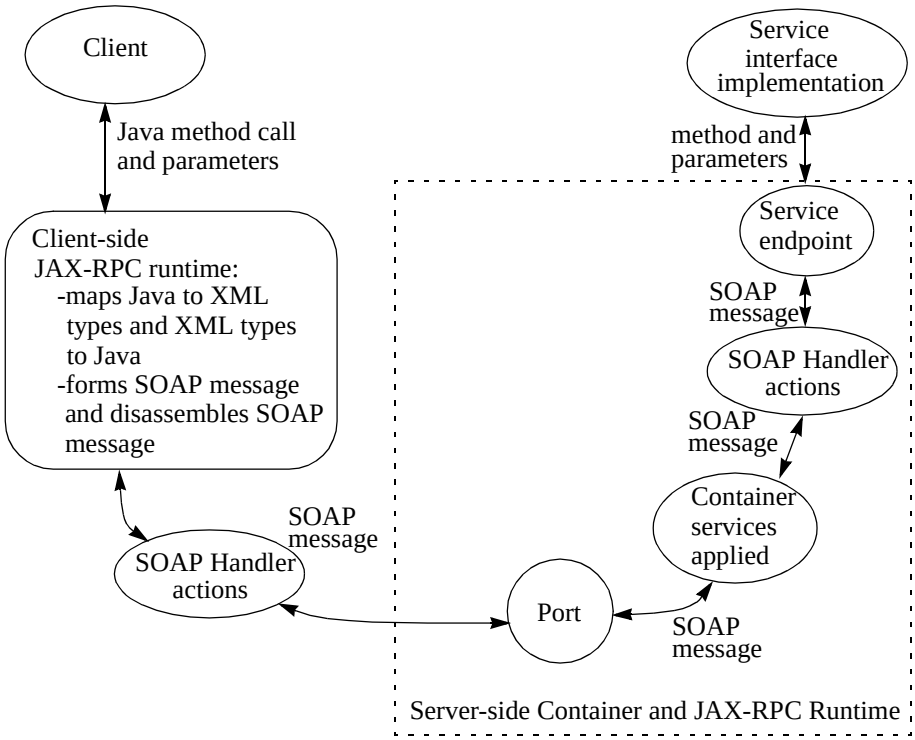

Figure 3.1 Structure of a Web Service

Once the client knows how to access the service, the client makes a request to the service by invoking a Java method, which is passed with its parameters to the

client-side JAX-RPC runtime. The client is actually invoking an operation on the service. (These operations represent the different services of interest to clients.) The JAX-RPC runtime maps the Java types to standard XML types and forms a SOAP message that encapsulates the method call and parameters. The runtime then passes the SOAP message through the SOAP handlers, if there are any, and then to the server-side service port.

A port type is associated with a port, and a single port can encompass one or more endpoints. The client's request reaches the service through a port, since a port provides access over a specific protocol and data format at a network endpoint consisting of a host name and port number.

Before the port passes the request to the endpoint, it ensures that the container applies its declarative services to the SOAP request. After that, any developer-written SOAP handlers in place are applied to the request. SOAP handlers are optional code that lets developers apply application-level services, such as common logging, quality of services, and so forth. After the handlers operate on the SOAP message, the message is passed to the service endpoint.

The endpoint extracts the client method call and the parameters for the call, performs any XML to Java object mapping necessary, and hands the method to the Web service interface implementation for processing.

> *Note: All the details between the method invocation and response just described happen under the hood. The platform shields the developer from these details. Instead, the developer deals only with typical Java programming language semantics, such as Java method calls, Java types, and so forth.*

3.3 Key Web Services Design Decisions

Keeping the three example scenarios in mind (see "Example Scenarios" on page 28), let's examine the key Web services design issues. When designing a Web service, consider the work that typical Web services undertake and the issues they address. In general, a Web service:

- Exposes an interface that clients use to make requests to the service
- Makes a service available to partners and interested clients by publishing the service details
- Receives requests from clients

- Delegates received requests to appropriate business logic and processes the requests
- Formulates a response for the request

Considering these issues, start designing your Web service by devising suitable answers to these questions:

- How will clients make use of your services? Consider what calls clients may make, what might be the parameters of those calls, and the kinds of endpoints you are going to use for your Web service.
- How will your Web service receive client requests?
- Where will such common pre-processing, such as transformations, translations, and logging, be done?
- How will the request be delegated to business logic?
- How will the response be formed and sent back?
- How are you going to let clients know about your Web service? Are you going to publish your service in public registries, in private registries, or some way other than registries?

The following are the typical steps for designing a Web service.

1. **Decide on the interface for clients. Decide if and how to publish this interface.**

   You as the Web service developer start the design process by deciding on the interface your service makes public to clients. The interface should reflect the kinds of calls that clients will make to use the service. You should also consider the type of endpoints you want to use—EJB endpoints or JAX-RPC service endpoints—and when to use them. The Web service's business logic often determines the best endpoint type. You must also decide if you are going to use SOAP handlers. It is also important to ensure your service's interoperability.

   Next, you decide if you want to publish the service interface, and, if so, how to publish it. Publishing a service makes it available to clients. You can restrict the service's availability to clients you have personally notified about the service, or you can make your service completely public and register it with a registry.

2. **Consider how to receive and preprocess requests.**

   Once you've decided on the interface and how to make it available, you are ready to consider how to receive requests from clients. You need to design your service to not only receive a call that a client has made, but to do any necessary preprocessing to the request before applying the service's business logic. Depending on the nature of the requests and the service, applying certain design patterns can help make this process more effective.

3. **Determine how to delegate the request to business logic.**

   Once a request has been received (and preprocessed), then you are ready to delegate it to the service's business logic. There are both architectures and design patterns that help with this mapping.

4. **Decide how to process the request.**

   Next, the service processes a request. Again, there are design patterns that assist with designing the processing logic.

5. **Determine how to formulate and send the response.**

   Last, you must design how the service formulates and sends a response back to the client. It's best to keep these operations logically together and use appropriate design patterns. There are also considerations to be taken into account before sending the response to the client.

Before delving into the details of these design issues, it is helpful to view a service in terms of layers. Essentially, a service implementation can be seen as having two layers: an interaction and a processing layer. (See Figure 3.2.)

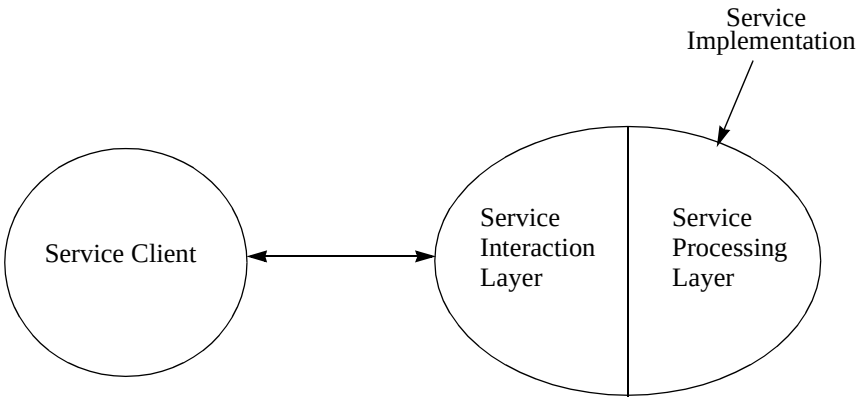

Figure 3.2 Layered View of a Web Service

The service interaction layer consists of the interface that the service exposes to clients and through which it receives client requests. The interaction layer also includes the logic for how the service delegates the requests to business logic and formulates responses. The interaction layer receives requests from clients through the endpoints and performs any preprocessing that might be required. The interaction layer then delegates requests to the business logic. When the business logic processing completes, the interaction layer sends back the response to the client.

The service processing layer holds all business logic used to process client requests. It is also responsible for integrating with EISs and other Web services.

Viewing your service implementation in this way helps to:

- Clearly divide responsibilities
- Provide a common or single location for request processing (both pre- and post-processing) logic in the interaction layer
- Expose existing business logic as a Web service

The remainder of this chapter discusses design issues for the layers of a service, and makes frequent references to suitable design patterns worth considering. For those new to this concept, design patterns give you a proven solution for a recurring problem in some defined context. This chapter does not discuss recommended design patterns in detail. You should refer to the design patterns section of the J2EE BluePrints Web site, at `http://java.sun.com/bluprints` for more information about patterns.

3.4 Designing a Service's Interaction Layer

A service's interaction layer has several major responsibilities, and chief among them is the design of the interface the service presents to the client. It is the interface through which clients access the service, and it is the start of a client's interaction with the service. The interaction layer also handles receiving client requests, mapping requests to appropriate business logic, and creating and sending responses.

3.4.1 Designing the Interface

Here are some considerations to keep in mind as you design the interface of your Web service.

3.4.1.1 Interface Definition Approaches

There are two approaches to developing the interface definition for a Web service:

1. Java to WSDL—Start with a set of Java interfaces for the Web service and from these create the WSDL description of the service for others to use.
2. WSDL to Java—Start with a Web Services Description Language (WSDL) document describing the details of the Web service interface and use this information to build the corresponding Java interfaces.

How do these two approaches compare? Starting with Java interfaces and creating a WSDL document is probably the easier of the two approaches. With this approach, you need not know any WSDL details because you use vendor-provided tools to create and publish the WSDL description. While these tools make it easy for you to generate WSDL files from Java interfaces, you do lose some control over the WSDL file creation. Different tools may have different interpretations for certain Java types, resulting in different representations in the WSDL file. On rare occasions, this may result in some semantic surprises.

The WSDL to Java approach requires a solid knowledge of WSDL and WS-I interoperability requirements. Although this approach gives you much more control over what the service exposes, even a small mistake can destroy the interoperability of your service. Since interoperability is one of the main forces for implementing a Web service, be very careful when using this approach. In short, while the WSDL to Java approach is more powerful, it requires more of developers and, as a result, is more difficult to use correctly.

3.4.1.2 Choice of the Interface Endpoint Type

In the J2EE platform, you have two choices for implementing the Web service interface—you can use a JAX-RPC service endpoint or an EJB service endpoint. Using one of these endpoint types makes it possible to embed the endpoint in the same tier as the service implementation. This simplifies the service implementation, because it obviates the need to place the endpoint in its own tier; the presence of the endpoint is solely to act as a proxy directing requests to other tiers.

Choosing which endpoint type to use for a Web service interface is straightforward when the business logic of the service is completely contained within either the Web tier or the EJB tier:

- *Use a JAX-RPC service endpoint when the processing layer is completely within the Web tier.*
- *Use an EJB service endpoint when the processing layer is only on the EJB tier.*

Of course, things aren't always this simple. When you are adding a Web service interface to an existing application or service, then you must consider if the existing application or service pre-processes requests before delegating them to the service's business logic. If so, then choose an endpoint type suited for the tier on which the pre-processing logic occurs—use a JAX-RPC service endpoint if the pre-processing occurs on the Web tier and an EJB service endpoint if pre-processing occurs on the EJB tier.

Given those general considerations, there are some additional points to keep in mind when choosing a Web service endpoint type. These points are:

- **Transaction considerations**—The transactional context in which a service implementation runs depends on the transactional context of the service implementation's container. This means that the transactional context is unspecified for a JAX-RPC service endpoint, because it runs in the Web container. Hence, if the Web service's business logic requires using transactions (and the service has a JAX-RPC endpoint), you must implement the transaction-handling logic using JTA or some other similar facility. When the Web service has an EJB endpoint, you as the developer do not need to write any code to handle transactions. An EJB service endpoint runs in the transaction context of an EJB container. The service's business logic thus runs under the transactional context as defined by the EJB container's `container-transaction` element in the deployment descriptor, and the container is responsible for handling transactions according to the setting of this element.
- **Method-level access permissions considerations**—A Web service's methods can be accessed by an assortment of different clients, and you may want to enforce different access constraints for each method. When you want to control service access at the method level, then consider using an EJB service endpoint rather than a JAX-RPC service endpoint. Enterprise beans permit method-level access permission declaration in the deployment descriptor—you can declare

various access permissions for different enterprise bean methods and the container correctly handles access to these methods. This holds true for an EJB service endpoint, since it is a stateless session bean. A JAX-RPC service endpoint, on the other hand, does not have a facility for declaring method-level access constraints. With a JAX-RPC endpoint, you must include handling access constraints in the code yourself.

- **HTTP session access considerations**—A JAX-RPC service endpoint, because it runs in the Web container, has complete access to an `HttpSession` object. Access to an `HttpSession` object, which can be used to embed cookies and store client state, may help you build session-aware clients. An EJB service endpoint, which runs in the EJB container, has no such access. However, because it is best to design Web services to be stateless, using `HttpSession` to store client state should only be done when absolutely necessary.

To illustrate, let's consider these endpoint type decisions from the standpoint of our sample scenarios. A heavily transactional Web service, such as an airline reservation service, might require varied access permissions for different methods, depending on the category of the user invoking the method. That is, users in frequent flyer or business traveller categories might have different access permissions from club member users or general users. In this case, it is a good design choice to implement the business logic using enterprise beans and expose the service through an EJB endpoint, thus leveraging the EJB container's transaction and method-level access permission services.

A weather information service, which provides read-only information and thus need not be transactional, also does not require the same degree of access permissions at the method level. For this type of service, it is a good choice to keep the service logic in enterprise beans and expose the service through an EJB endpoint. However, an equally valid design choice is to use a JAX-RPC service endpoint and keep the service logic in the Web tier. This latter choice is handier when the service implementors do not have EJB component skills.

3.4.1.3 Granularity of Service

Much of the design of a Web service interface involves designing the service's operations, or its methods. You first determine the service's operations, then define the method signature for each operation. That is, you define each operation's parameters, its return values, and any errors or exceptions it may generate.

You should keep the same considerations in mind when designing the methods for a Web service as when designing the methods of a remote enterprise bean. This is particularly true not only regarding the impact of remote access on performance, but also with Web services, it is important because there is an underlying XML representation requiring parsing and taking bandwidth. Thus, a good rule is to define the Web service's interface for optimal granularity of its operations; that is, find the right balance between coarse-grained and fine-grained granularity. Generally, you should consolidate related fine-grained operations into more coarse-grained ones to minimize expensive remote method calls. More coarse-grained service operations, such as returning catalog entries in sets of categories, keep network overhead lower and improve performance. However, they are often less flexible from a client's point of view. While finer-grained service operations, such as browsing a catalog by products or items, offer a client greater flexibility, these operations result in greater network overhead and reduced performance.

Keep in mind, also, that too much consolidation leads to inefficiencies. For example, consolidating logically different operations is inefficient and should be avoided. Consolidating similar operations, such as querying operations, is much better.

Good design for our airline reservation Web service, for example, is to expect the service's clients to send all information required for a reservation—destination, preferred departure and arrival times, preferred airline, and so forth—in one invocation to the service, that is, as one large message. This is far more preferable than to have a client invoke a separate method for each piece of information comprising the reservation. However, it might not be a good idea to combine in a single service invocation the same reservation with an inquiry method call.

Along with optimal granularity, you should consider data caching issues. Coarse-grained services involve transferring large amounts of data. As a result, if you opt for more coarse-grained service operations, it is more efficient to cache data on the client side to reduce the number of round trips between the client and the server. On the other hand, fine-grained services, which require more back and forth interaction between the client and the server, perform more efficiently if you use data caching on the server side.

Good design for a weather information service, for example, would be to have the service plan to cache at the server side weather information served to a client. Doing so might obviate the need to make repeated trips to the processing layer for frequently accessed weather information.

3.4.1.4 Interfaces and Parameters

There are some points to consider regarding parameter handling and passing.

3.4.1.4.1 Java Objects as Parameters

When designing parameters for method calls in a Web service interface, choose parameters that have standard mapping types. (See Figure 3.3.) Note that the JAX-RPC-supported types are as follows:

- Built-in types—includes all primitives and wrapper types: `int`, `Integer`, `String`, `Date`, `Calendar`, `BigInteger`, `BigDecimal`, and `Qname`
- JAX-RPC value types—user-defined classes, including classes with JavaBeans™ component-like properties and `public` fields

Keep in mind that the portability of your service is reduced when you use vendor-provided serialization and deserialization techniques for types not supported by default. (See Section 3.4.1.4.3 on page 39 for more details.)

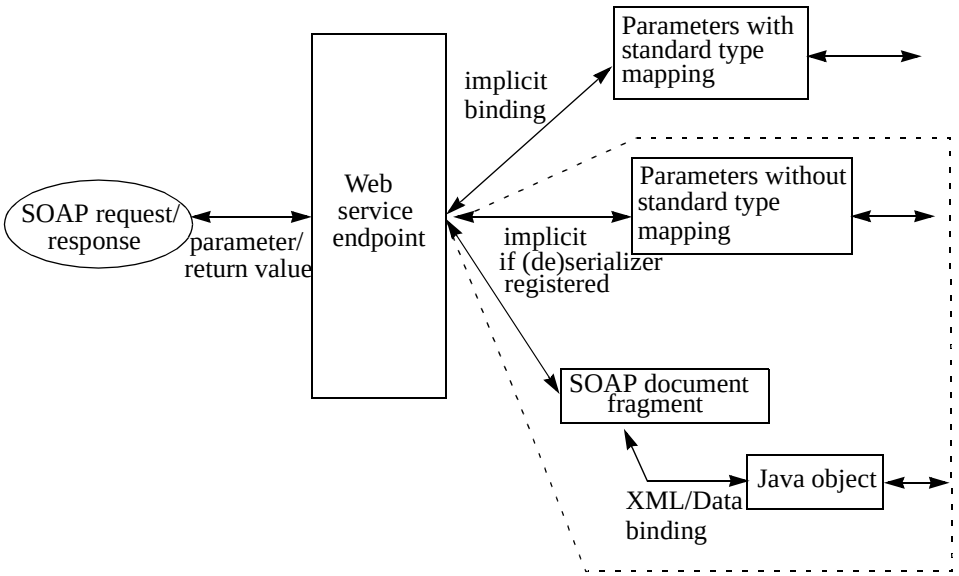

Figure 3.3 Binding Parameters and Return Values with JAX-RPC

3.4.1.4.2 XML Documents as Parameters

There are cases when you want to pass XML documents as parameters. See "Handling Documents in Business Process Services" on page 51 for guidelines.

3.4.1.4.3 Handling Non-Standard Type Parameters

JAX-RPC technology, in addition to providing a rich standard mapping set between XML and Java data types, also provides an extensible type mapping framework. Developers can use this framework to specify pluggable, custom serializers and deserializers that support non-standard type mappings.

However, this extensible type mapping framework is not yet a standard part of the J2EE platform, and vendors can provide their own solutions to this problem. It must be emphasized that if you implement a service using some vendor's implementation-specific type mapping framework, then your service is not guaranteed to be portable. Because of these portability limitations, you should avoid passing parameters that require the use of vendor-specific serializers or custom deserializers.

Parameters whose types do not have registered serializers may be passed as SOAP document fragments constituting a DOM subtree. (See Figure 3.3.) If so, you should as much as possible (either manually or using JAXB) bind the SOAP fragments to Java objects before passing them to the processing layer.

3.4.1.5 Use of Handlers

As discussed in Chapter 2, JAX-RPC technology enables you to plug in SOAP message handlers, thus allowing processing of SOAP messages that represent requests and responses. This gives you the capability to examine and modify the SOAP requests before they are processed by the Web service and to examine and modify the SOAP responses before they are delivered to the client. Keep in mind, however, that these handlers are specific to only SOAP requests and responses.

Handlers are particular to a Web service and are associated with the specific port of the service. As a result of this association, the handler's logic applies to all SOAP requests and responses that pass through a service's port. Thus, you use these message handlers when your Web service must perform some SOAP message-specific processing common to all its requests and responses. It is not advisable to put business logic or processing particular to requests and responses in a handler.

Furthermore, you cannot store client-specific state in a handler. From a client point of view, handlers must be stateless. However, you may use the handler to

store port-specific state, which is state common to all method calls on that service interface. Note, also, that handlers run in the context of the component in which they run.

Use of handlers could potentially affect the interoperability of your service. See the next section on interoperability. Keep in mind that it takes advanced knowledge to correctly use handlers. As such, Web service developers should try to use existing or vendor-supplied handlers, such as for handling auditing, logging, and so forth, as much as possible.

3.4.1.6 Interoperability

Probably the major benefit of Web services is interoperability between heterogeneous platforms. To get the maximum benefit, you want to design your Web service to be interoperable with clients on any platform, and, as discussed in Chapter 2, the Web Services Interoperability (WS-I) organization helps in this regard. WS-I promotes a set of generic protocols for the interoperable exchange of messages between Web services. The WS-I basic profile promotes interoperability by defining and recommending how a set of core Web services specifications and standards (including SOAP, WSDL, UDDI, and XML) can be used for developing interoperable Web services.

In addition to the WS-I protocols, organizations such as SOAPBuilder.org, by providing common testing grounds, have made it possible for various Web services technology vendors to test the interoperability of implementations of their standards. When you implement your service using technologies that have passed this interoperability testing, you are assured that such services are interoperable.

Since WS-I supports only literal bindings, you should avoid encoded bindings. Literal bindings also cannot represent complex types, such as a network of objects, with code references. For interoperability, you should keep both these limitations in mind.

Also note that handlers which give you access to SOAP messages at the same time impose major responsibilities on you. You must be careful not to change a SOAP message to the degree that the message no longer complies with WS-I specifications.

3.4.2 Receiving Requests

The interaction layer, through the endpoint, receives client requests. Incoming client requests (in the form of SOAP messages) are mapped to method calls on the classes

that implement the Web service interface. These classes should perform security validation, transform parameters, and pre-process the parameters for these requests before delegating the requests to the Web service business logic.

Parameters are passed as either Java objects, XML documents (`javax.xml.transform.Source` objects), or even SOAP document fragments (`SOAPElement` objects). While it is straightforward to handle parameters bound to Java objects, additional steps may be required with XML documents. (See XML chapter for more details.) The following steps are recommended:

1. The service endpoint should validate the incoming XML document against its schema.
2. When the service is designed to deal with XML documents, you should transform the XML document to an internally supported schema, if the schema for the XML document differs from the internal schema, before passing the document to the processing layer.
3. When the service implementation deals with objects, then, as part of the interaction layer, map the incoming XML documents to domain objects.

It is important that these three steps—validation of incoming parameters or XML documents, translation of XML documents to internal supported schemas, and mapping documents to domain objects—be performed as close to the service endpoint as possible, and certainly in the service interaction layer. Catching errors earlier avoids unnecessary calls and round-trips to the processing layer.

Figure 3.4 shows the recommended way to handle requests and responses in the Web interaction layer.

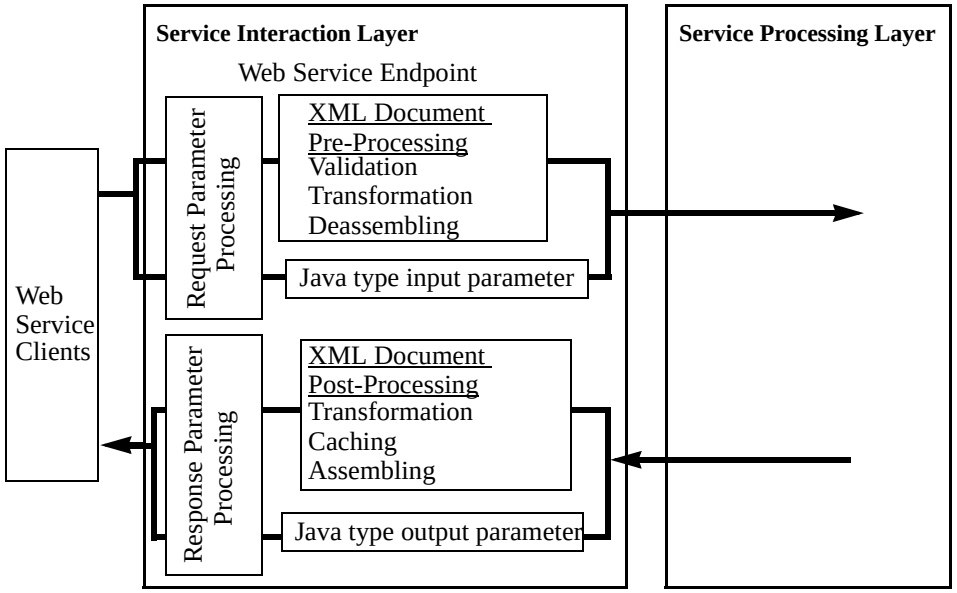

Figure 3.4 Web Tier Request and Response Processing

The Web service endpoint handles all incoming requests and delegates them to the business logic exposed in the processing layer. The Web service interface when implemented in this manner has several advantages, since it gives you a common location for the following tasks:

- Managing the handling of requests so that the service endpoint serves as the initial point of contact
- Invoking security services, including authentication and authorization
- Validating and transforming incoming XML documents and mapping XML documents to domain objects
- Delegating business processing
- Handling errors

As examination of the sample scenarios indicates, it is generally advisable to do all common processing—such as security checks, logging, auditing, input vali-

dation, and so forth—for requests at the interaction layer as soon as a request is received and before passing it to the processing layer.

3.4.3 Delegating Web Service Requests to Processing Layer

After designing the request processing tasks, the next step is to design how the request is going to be delegated to the processing layer. At this point, consider the kind of processing the request requires, since this helps you decide how to delegate the request to the processing layer. All requests can be categorized into two large categories based on the time it takes to process the request, namely:

- A request that is processed in a short enough time so that a client can afford to block and wait to receive the response before proceeding further. In other words, the client and the service interact in a synchronous manner such that the invoking client blocks until the request is processed completely.
- A request that takes a long time to be processed, so much so that it is not a good idea to make the client wait until the processing is completed. In other words, the client and the service interact in an asynchronous manner such that the invoking client need not block and wait until the request is processed completely.

Note: When referring to request processing, we use the terms synchronous and asynchronous from the point of view of when the client's request processing completes fully. Keep in mind that, under the hood, an asynchronous interaction between a client and a service might result in a synchronous invocation over the network, since HTTP is by its nature synchronous. Similarly, SOAP messages sent over HTTP are also synchronous.

The weather information service is a good example of a synchronous interaction between a client and a service. When it receives a client's request, the weather service must look up the required information and send back a response to the client. This look up and return of the information can be achieved in a relatively short time, during which the client can be expected to block and wait. The client continues its processing only after it obtains a response from the service. (See Figure 3.5.)

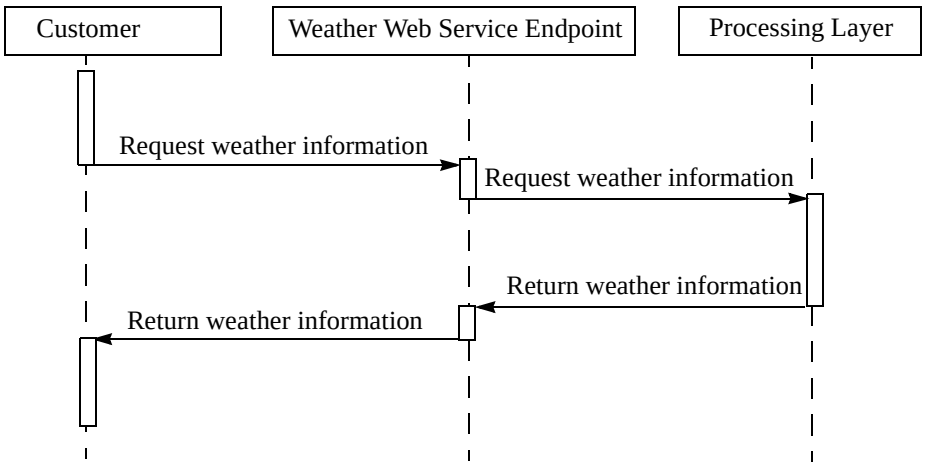

Figure 3.5 Weather Information Service Interaction

A Web service such as this can be designed using a service endpoint that receives the client's request and then delegates the request directly to the service's appropriate logic in the processing layer. The service's processing layer processes the request and, when the processing completes, the service endpoint returns the response to the client. (See Figure 3.6.)

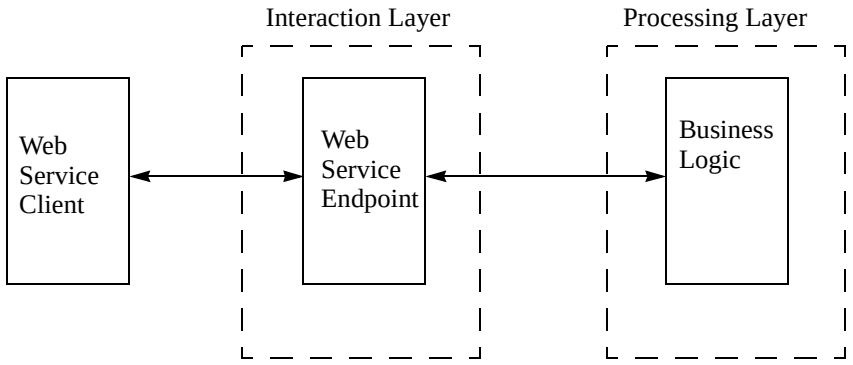

Figure 3.6 Synchronous Interaction Between Client and Service

Now let's examine an asynchronous interaction between a client and a service. When making a request for this type of service, the client cannot afford to

wait for the response because of the significant time it takes for the service to process the request completely. Instead, the client may want to continue with some other processing. Later, when it receives the response, the client resumes whatever processing initiated the service request. Typically in these types of services, the content of the request parameters initiates and determines the processing workflow—the steps to fulfill the request—for the Web service. Often, fulfilling a request requires multiple workflow steps.

The travel agency service is a good example of an asynchronous interaction between a client and a service. A client requests arrangements for a particular trip by sending the travel service all pertinent information (most likely in an XML document). Based on the document's content, the service performs such steps as verifying the user's account, checking and getting authorization for the credit card, checking accommodations and transportation availability, building an itinerary, purchasing tickets, and so forth. Since the travel service must perform a series of often time-consuming steps in its normal workflow, the client cannot afford to pause and wait for these steps to complete.

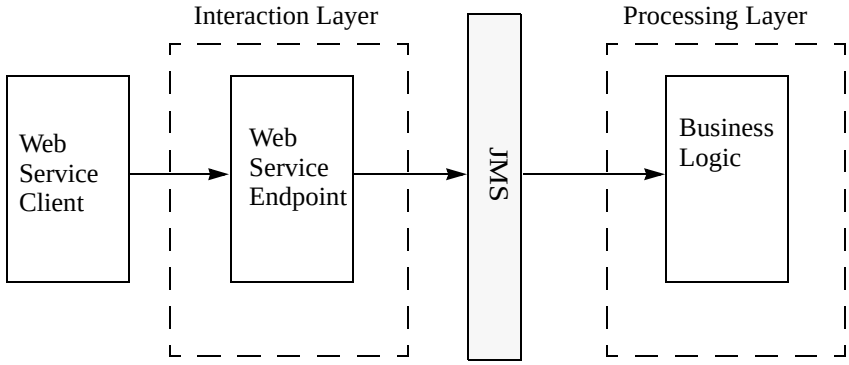

Figure 3.7 Asynchronous Interaction Between Client and Service

Figure 3.7 shows one recommended approach for delegating these types of Web service requests to the processing layer. In this architecture, the client sends a request to the service endpoint. The service endpoint delegates the client's request to the appropriate processing layer of the service. It does so by sending the request as a JMS message to a JMS queue or topic specifically designated for this type of request.

After the request is successfully delegated to the processing layer, the service endpoint may return a correlation ID to the client. (This correlation ID helps the client recognize a response that corresponds to a previous request.) Message-driven beans in the EJB tier read the request and initiate processing so that a response can ultimately be formulated. The service may make the result to the client's request available in one of two ways:

- The client that invoked the service periodically checks the status of the request using the correlation ID that was provided at the time the request was submitted. (This is also known as polling.)
- Or, if the client itself is a Web service peer, the service calls back the client's service with the result. The client may use the correlation ID to relate the response with the original request.

When delegating service requests, consider using these design patterns:

- Service Locator pattern—This pattern abstracts the process of looking up, or locating, a resource. Consider using it in both the interaction and processing layers of your Web service.
- Business Delegate pattern—This pattern helps to reduce tight coupling between a service's layers.

3.4.4 Formulating Responses

You should perform response generation, which is simply constructing the method call return values and output parameters, on the interaction layer, as close as possible to the service endpoint. This permits having a common location for response assembly and XML document transformations, particularly if the document you return to the caller must conform to a different schema from the internal schema. Keeping this functionality in the endpoint lets you implement data caching and avoid extra trips to the processing layer. (See Figure 3.4 on page 42.)

Consider response generation from the weather information service's point-of-view. The weather information service may be used by a variety of client types, from browsers to rich clients to hand-held devices. A well-designed weather information service would render its responses in formats suitable for these different client types. However, it is not good design to have a different implementation of the service's logic for each client type. Thus, it is important to consider the

above guidelines when you have a service that has a common processing logic but potentially with different response rendering needs to fit its varied client types.

3.5 Processing Layer Design

The processing layer is where the business logic is applied to a Web service request. The issues and considerations for designing an application's processing or business logic layer, such as whether to perform this logic in the Web or EJB tier, are the same whether or not you use a Web service. That is, regardless of the means you use to expose your application's functionality, the business logic design issues are the same. You must still design the business logic by considering such issues as using enterprise beans, exposing a local or a remote client interface model, using container-managed or bean-managed persistence, and so forth.

We do not address these business logic design issues here, since much of this discussion has already been covered in the book *Designing Enterprise Applications with the J2EE™ Platform, Second Edition*, and you scan refer to that book for general guidelines and recommendations. You should also refer to the BluePrints Web site at `http://java.sun.com/blueprints`, for recommendations on designing an application's business processing logic. You should also review the J2EE patterns (available at the BluePrints Web site), since a number of these patterns may be applicable for the design of the processing layer.

In addition to these general guidelines, there are some specific issues to keep in mind when designing the processing layer of a Web service.

- **Keep the processing layer independent of the interaction layer**—By keeping the layers independent and loosely coupled, the processing layer remains generic and can support different types of clients, such as Web service clients, classic Web clients, and so forth. To achieve loose coupling between the layers, consider using the business delegate design pattern as a guide if it is applicable to your application scenario.
- **Bind XML documents to Java objects in the interaction layer**—There are times when your Web service expects to receive from a client an XML document containing a complete request, but the service's business logic has no need to operate on the document. On these occasions, it is recommended that the interaction layer bind the XML document contents to Java objects before passing the request to the processing layer. Since the processing logic does not have to perform the XML to Java conversion, a single processing layer can

support XML documents that rely on different schemas.

Keep in mind that your processing logic can operate on the contents of an XML document received from a client. Refer to "Handling Documents in Business Process Services" on page 51, which highlights issues to consider when you pass XML documents to your business processing logic.

Depending on your application scenario, your processing layer may be required to work with other Web service peers to complete a client's request. If so, your processing layer effectively becomes a client of another Web service. Refer to Chapter 5 for guidelines on Web service clients. In other circumstances, your processing layer may have to interact with EISs. For these cases, refer to <xref integration chapter> for guidelines.

You may also find that your Web service's processing logic raises other issues, such as distributed transaction handling, integration of multiple business processes, and so forth. See "Advanced Issues" on page 87 for a discussion of these issues.

3.6 Publishing a Web Service

Up to now, this chapter has covered guidelines for designing and implementing your Web service. A Web service may be for use by the general public, between enterprises (inter-enterprise), or just within an enterprise (intra-enterprise). For this to happen, the Web service endpoint must be made accessible to clients by publishing a description of the Web service. *A Web service description consists of not only the service's Web Services Description Language (WSDL) description, but also the XML schemas referenced by the service description.* (As already noted, WSDL is the standard language for describing a service.)

For interoperability, your Web service must publish its public XML schemas and any additional schemas defined in the context of the service. You must register Web services for general public consumption on a well-known public registry. When a Web service is strictly for intra-enterprise use, you may publish a Web service description on a corporate registry within the enterprise. You also *must* publish XML schemas referred to by the Web service description on the same corporate or public repository. A Java-based client uses the JAXR APIs to look up the description of any Web service on a corporate or public registry.

You do not need to use a registry if all the customers of your Web services are dedicated partners and there is an agreement among the partners on the use of the

services. In this case, you can publish your Web service description (the WSDL and referenced XML schemas) at a well-known location with the proper security access protections. An example might be a client application of a reseller that has an agreement with a particular supplier and has been statically bound at development time to its supplier's Web service. Only authorized parties can retrieve the service description to generate the client code, as well as reference the XML schemas.

A registry is not a data repository.

Keep in mind that public registries are not repositories, in the sense that they do not contain complete details on services. For example, a service selling pets does not register its complete catalog of products. The registry maintains details on the type of service, its location, and how to access the service, plus other pieces of information required to access a service. The client must discover and bind with the service to get the service's complete catalog of products. For example, a client interested in purchasing a cat might use a registry to discover a Web service that sells pets. However, when the client actually accesses the service, it finds out that this particular service does not sell cats. The client goes back to the registry to find another pet seller service. When a client does find a service that meets its specific requirements, the client should cache the Web service location to avoid repeated look ups at a later time.

Therefore, it is important to register your service under the proper taxonomies. When you want to publish your service on a registry, either a public or corporate registry, you must do so against a taxonomy that correctly classifies or categorizes your Web service. It is important to decide on the proper taxonomy, as this affects the ease with which clients can find and use your service. Several well-defined industry standard taxonomies exist today, such as those defined by organizations such as the North American Industry Classification System (NAICS).

Register under the proper taxonomy.

Using existing, well-known taxonomies gives clients of your Web service a standard base from which to search for your service, making it easy for clients to find your service. For example, suppose your business provides a stock and mutual fund quote service. Rather than create your own taxonomy to categorize your service, clients will more easily find your service if you publish your service description using two different standard taxonomies for stock and fund quotes. To further help clients find your service, it is also a good idea to publish in as many

applicable categories as possible. For example, if your business sells particular products, register using a product category taxonomy as well as a geographical taxonomy. This gives clients a chance to use optimal strategies for locating a service. For example, if multiple instances of services exist for a particular product, the client can further refine its selection by considering geographical location and choosing a service close to its own location.

When a registry is used, the service provider publishes the Web service description on a registry and clients discover and bind to the Web service to use its services. In general, a client must perform three steps to use a Web service:

1. The client must determine how to access the service's methods, such as determining the service method parameters, return values, and so forth. This is referred to as discovering the service definition interface.
2. The client must locate the actual Web service; that is, find the service's address. This is referred to as discovering the service implementation.
3. The client must be bound to the service's specific location, and this may occur on one of three occasions:
   - When the client is developed (called static binding)
   - When the client is deployed (also called static binding)
   - During run time (called dynamic binding)

These three steps may produce three scenarios, depending on whether the client is generic or implemented for a specific service and when the binding occurs. The following paragraphs describe these scenarios (see Table 3.1 for a summary). They also note important points you should consider when designing and implementing a Web service. (Chapter 5 considers these scenarios from the point of view of a client.)

- Scenario 1: The Web service has an agreement with its partners and publishes its WSDL description and referenced XML schemas at a well-known, specified location. It expects its client developers to know this location. When this is the case, the client is implemented with the service's interface in mind. When it is built, the client is already designed to look up the service interface directly rather than using a registry to find the service.

- Scenario 2: Similar to scenario 1, the Web service publishes its WSDL description and XML schemas at a well-known location, and it expects its partners to

either know this location or be able to discover it easily. The partner, for its part, is implemented and built with the service location incorporated. Or, when the partner is built, it can use a tool to dynamically discover and then include either the service's specific implementation or the service's interface definition, along with its specific implementation. In this case, binding is static because the partner is built when the service interface definition and implementation are already known to it, even though this information was found dynamically.

- Scenario 3: The service implements an interface at a well-known location, or it expects its clients to use tools to find the interface at build time. Since the Web service's clients are generic clients—they are not clients designed solely to use this Web service—you must design the service so that it can be registered in a registry. Such generic clients dynamically find a service's specific implementation at run time using registries. Choose the type of registry for the service—either public, corporate, or private—depending on the types of its clients—either general public or intra-enterprise—its security constraints, and so forth.

Table 3.1 Discovery-Binding Scenarios for Clients

Scenarios	Discover Service Interface Definition	Discover Service Implementation	Binding to Specific Location
1	None	None	Static
2	None or dynamic at build time	Dynamic at build time	Static or dynamic at run time
3	None or dynamic at build time	Dynamic at run time	Dynamic at build time

3.7 Handling Documents in Business Process Services

Up to now, the chapter addressed issues applicable to all Web service implementations. There are additional considerations when a Web service implementation expects to receive an XML document containing all the information from a client, and which the service uses to start a business process to handle the request.

For example, consider the travel agency Web service, which typically receives a client request as an XML document containing all information needed to arrange

a particular trip. The information in the document includes details about the customer's account, credit card status, desired travel destinations, preferred airlines, class of travel, dates, and so forth. The Web service uses the documents contents to perform such steps as verifying the customer's account, obtaining authorization for the credit card, checking accommodations and transportation availability, building an itinerary, and purchasing tickets.

In essence, the service, which receives the request with the XML document, starts a business process to perform a series of steps to complete the request. The contents of the XML document are used throughout the business process. Handling this type of scenario effectively requires some additional considerations to the general ones for all Web services.

- Good design expects XML documents to be received as javax.xml.transform.Source objects. See "Receiving XML Documents as Parameters," which discusses XML documents as parameters.
- It is good design to do the validation and any required transformation of the XML documents as close to the endpoint as possible. Validation and transformation should be done before applying any processing logic to the document content. See Figure 3.4 on page 42 and the discussion on receiving requests in Section 3.3 on page 30.
- It is important to consider the processing time for a request and whether the client waits for the response. When a service expects an XML document as input and starts a lengthy business process based on the document contents, then clients typically do not want to wait for the response. Good design when processing time may be extensive is to delegate a request to a JMS queue and return an identifier for the client's future reference. (Recall Figure 3.7 on page 45 and its discussion.)

The following sections discuss other considerations.

3.7.1 Receiving XML Documents as Parameters

There are times when you may have to receive XML documents as parameters. For example, a client of the example travel agency service typically sends to the service a request with all information regarding a planned trip—cars, hotels, flights, credit card—and the service starts a business process to fulfill this request. In such instances, the client should be expected to wrap the XML document specifying the

request into a `javax.xml.transform.Source` object. You can also pass an URL to the XML document.

The Web service endpoint can expect to receive documents as arguments (such documents are wrapped in `Source` objects), plus other JAX-RPC arguments containing metadata, processing requirements, security information, and so forth. When an XML document is wrapped as a `Source` object before being sent across the network, you are freed from the intricacies of sending and retrieving documents as part of request/response handling. The `Source` object lets the container implementation handle these document-passing details, which it is designed to do.

3.7.2 Abstract Document Manipulation From Processing Logic

It's good practice to abstract document manipulation logic from the processing logic. Since a Web service starts a business process based on the contents of an XML document, the business processing logic must at a minimum read the document, if not modify the document. By separating the document manipulation logic from the processing logic, a developer can switch between various document manipulation mechanisms without affecting the processing logic. In addition, there is a clear division between developer skills.

> *It is a good practice to separate the XML document manipulation logic from the business logic.*

Chapter 4 provides more information on how to accomplish this separation and its merits.

3.7.3 Fragment XML Documents

Generally, it is a good idea to break XML documents into logical fragments when appropriate. The processing logic receives an XML document containing all information for processing a request. However, the XML document usually has well-defined segments for different entities, and each segment contains the details about a specific entity.

> *Rather than pass the entire document to different components handling various stages of the business process, it's best if the processing logic breaks the document into fragments and passes only the required fragments to other components or services that implement portions of the business process logic.*

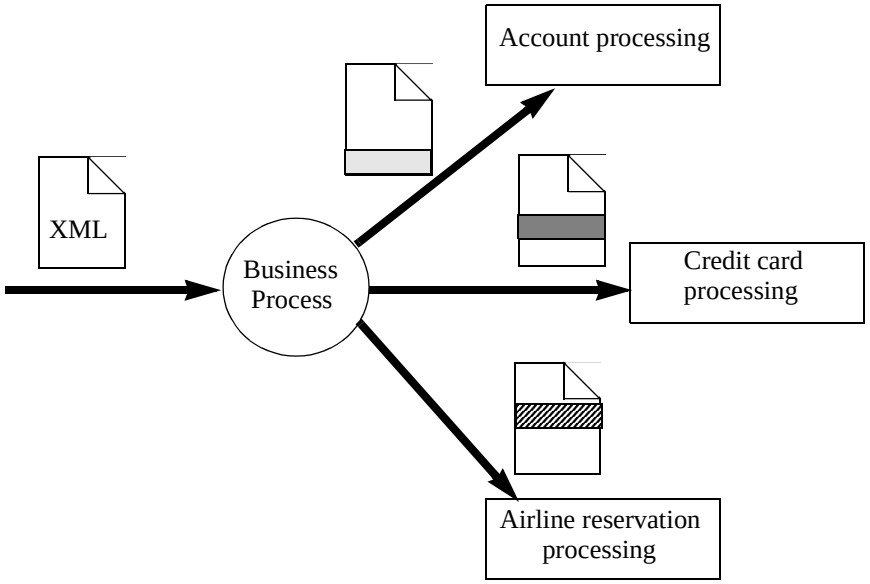

Figure 3.8 Fragmenting an XML Document

Figure 3.8 shows how the processing layer might process an XML document representing an incoming XML document for our travel agency Web service. The document contains such details as account information, credit card data, preferred travel destinations, dates and so forth. The business logic involves verifying the account, authorizing the credit card, and filling the airline, hotel, and rental car portions of the travel itinerary. It is not necessary to pass all the document details to a business process stage that is only performing one piece of the business process, such as account verification. Passing the entire XML document to all stages of the business process results in unnecessary information flows and extra processing. It is more efficient to extract the logical fragments—account fragment, credit card fragment, and so forth—from the incoming XML document and then pass these individual fragments to the appropriate business process stages in the an appropriate format (DOM tree, Java object, serialized XML, and so forth) expected by the receiver.

Fragmenting a document has the following benefits:

- It avoids extra processing of superfluous information throughout the workflow.
- It maximizes privacy because it limits sending sensitive information through the workflow.
- It centralizes the workflow and simplifies the workflow implementation.
- It provides greater flexibility to workflow error handling.

3.7.4 Using XML

XML, while it has many benefits, also has performance disadvantages. You should refer to Chapter 4, which provides guidelines on XML processing. Following these guidelines may help minimize the performance overhead of using XML. When deciding on an approach for using XML, keep in mind the costs involved for using XML and weigh them along with the recommendations on parsing, validation, and binding documents to Java objects. The rationale behind these recommendations is discussed, and you should consider this, too, when deciding on your approach.

Additional guidelines apply for remote interaction modes—that is, when handling XML documents for those business processes whose implementation spans multiple containers. For this remote interaction, it is expensive to pass DOM trees. While it is expensive and thus best avoided, using serialized XML may be required to provide loosely-coupled integration points. Instead, consider passing Java objects, as that might be the best choice in this scenario.

When business process components follow a local interaction mode—they reside in the same container or virtual machine—then it is highly efficient to pass Java objects. It is also efficient to pass DOM trees by reference, though this may not be the optimal choice from the perspective of the business object interface. Because DOM trees require a certain amount of manipulation, passing DOM trees by reference impacts memory and processing. However, local components should not pass serialized XML among themselves. In the context of a local call, there is no reason to serialize XML and later deserialize it.

3.7.5 Use JAXM and SAAJ Technologies for Handling Documents

The J2EE platform provides an array of technologies—including mandatory technologies such as JAX-RPC and SAAJ and optional technologies such as Java™ API

for XML Messaging (JAXM)—that enable message and document exchanges with SOAP. Each of these J2EE technologies offers a different level of support for SOAP-based messaging and communication. (See Chapter 2 for the discussion on JAX-RPC and SAAJ.)

An obvious question that arises is: Why not use JAXM or SAAJ technologies in scenarios where you have to pass XML documents? If you recall:

- SAAJ lets developers deal directly with SOAP messages, and is best suited for point-to-point messaging environments. SAAJ is better for developers who want more control over the SOAP messages being exchanged.
- JAXM defines an infrastructure for guaranteed delivery of messages. It provides a way of sending and receiving XML documents and guaranteeing their receipt, and is designed for use cases that involve storing and forwarding XML documents and messages.

SAAJ is considered more useful for advanced developers who thoroughly know the technology and who must deal directly with SOAP messages. However, SAAJ is not recommended with services that require you to pass XML documents.

Using JAXM for scenarios that require passing XML documents may be a good choice. Note, though, that JAXM is optional in the J2EE 1.4 platform. As a result, a service developed with JAXM may not be portable.

3.8 Conclusion

This chapter began with a description of Web service fundamentals. It described the underlying structure of a typical Web service, showing how the various components making up clients and services pass requests and responses among themselves. The chapter also described some example scenarios, which it used to illustrate various concepts. Once the groundwork was set, the chapter discussed the key design decisions that a Web service developer needs to make, principally the design of a service as an interaction and a processing layer. It traced how to go about making these design decisions and recommending good design choices for specific scenarios.

The next chapter focuses on developing Web service clients.